Ralypto Gym: Master Agent Prompt (2-Day Build)

How to use: copy everything between **START OF PROMPT** and **END OF PROMPT** into your coding agent in an empty folder. The prompt is self-contained (the agent does not need the plan doc). If the agent stops or runs out of room, send the short "Continue" prompts at the end.

Honest Expectations (Read First)

- **Two days with an agent can give you a demo-ready, working MVP on seeded data**: real backend, real database, real login, all hero AI features working. That is a strong thing to show gym owners.
- **It will not be a finished, production-ready product.** Still needed before real gyms pay: a test in one real gym, a payment gateway, WhatsApp/SMS provider approval, a security review, hosting and backups, and legal checks for health data.
- **Agents often say "done" too early.** The prompt forces them to run tests and give an honest report. Always verify with the 10-step click path at the end.
- **Your job in these 2 days:** review each stage, run the app, and test like a gym front-desk person. Do not just trust the report.
- In this version the agent builds **both frontend and backend** (one FastAPI app, with AI inside it). If your friend still does the backend, tell the agent to stop after Stage 0 and use the generated API spec as the contract.

START OF PROMPT

You are a senior full-stack engineer and product designer. Build **Ralypto Gym**, a gym management system sold to gym owners as a monthly subscription, in **2 days of work**. Work through the Stages below **without stopping to ask me questions**. Pick sensible defaults and write them in `docs/DECISIONS.md`. Commit after each Stage. Never push.

0. Product in One Paragraph

Gym owners manage members, memberships, payments, check-in, classes, trainers and workouts in one clean system. Members use a phone app (PWA) for QR check-in, class booking, workouts and progress. **AI makes it special:** it finds members about to quit and drafts win-back messages, builds personal workout and meal plans (trainer approves), answers members by chat and voice, forecasts how busy the gym is, and lets the owner ask questions about the business. Languages: English, Sinhala, Tamil. Currency: LKR.

1. Rules (Important)

1. **Be honest.** Never say something is done unless you ran it and saw it work. Say what is mocked.
2. **Easy for non-technical staff:** big buttons (48px+), icon plus text label, simple words ("Renew", "Check In", "Add Member"), max 3 taps for daily tasks, one main button per screen.
3. **Style: Apple-like, black and white.** White and light grey backgrounds (#ffffff, #f5f5f7), near-black text (#1d1d1f), black primary pill buttons, 18px rounded cards, hairline borders, almost no shadow, thin lucide icons, calm 200-300ms motion (fade plus small slide), no bouncing. Colours only for small status marks: green OK, amber expiring or waiting, orange busy, red expired or error, blue info. Font: Inter. Light mode default, dark mode (pure black) available.
4. **Every screen has 4 states:** loading (skeleton), empty (friendly text and one button), error (plain words and Retry), normal.
5. **Multi-tenant from day 1:** every table has `gym_id`; every query filters by it. Write a test that proves one gym cannot see another's data.
6. **Money in integer cents** in the database. Format with one `formatMoney` helper (LKR).
7. **AI rules:** AI proposes, a human confirms anything important. AI never writes to the database directly; it calls the same service functions as a normal user, with that user's permissions. AI never invents facts (class times, prices, balances come from the database). Every AI action is saved in `ai_actions` (who asked, tool, result, time). Nothing medical: no medical advice, no body-fat or health estimates from photos. If AI or the API key is missing, everything else must still work (use the built-in fallbacks).
8. **Secrets:** no real keys in code. Use `.env` and `.env.example`.
9. After each Stage run lint, type-check, tests and build, and fix errors.

2. Stack (Decided)

- **Monorepo** `ralypto-gym/`: `frontend/`, `backend/`, `docs/`, `docker-compose.yml`, `README.md`.
- **Frontend:** React 18 + TypeScript + Vite, Tailwind, React Router, TanStack Query, Zustand, framer-motion, lucide-react, react-hook-form + zod, recharts, sonner, date-fns, react-i18next, vite-plugin-pwa, qrcode.react, html5-qrcode (camera scan), Dexie (offline), @tanstack/react-virtual (long lists), @fontsource-variable/inter, openapi-typescript (generate API types from the backend OpenAPI file). Vitest for tests. One app with two areas: **Staff app** (`/app/*`) and **Member app** (`/m/*`, mobile-first PWA).
- **Backend:** Python 3.11+, FastAPI, SQLModel (SQLAlchemy), Pydantic v2, JWT (access plus refresh), bcrypt. **SQLite by default**, PostgreSQL by changing `DATABASE_URL`. Pytest. The AI code lives in `backend/app/ai/`.
- **LLM:** put it behind an `LLMClient` interface with two implementations: `GeminiClient` (uses `GEMINI_API_KEY` and `LLM_MODEL` from env; check the current SDK docs for a valid model name and do not hard-code one) and `MockLLM` (deterministic, template-based answers so demos work with no key). Default to `MockLLM` when no key exists.
- **Messaging:** `MessageProvider` interface with `LogProvider` (default: saves to an outbox table and shows it in the app) and a `wa.me` click-to-chat link for WhatsApp. Real SMS/WhatsApp providers come later.
- **Voice:** browser Web Speech API for speech-to-text where supported (language selector EN / SI / TA), with typing as the fallback. Add a clear note: Sinhala and Tamil voice accuracy depends on the browser and must be tested.

3. Roles and Plans

Roles: `owner`, `manager`, `front_desk`, `trainer`, `member`. Role checks happen in the backend (a `require_role` dependency). The frontend only hides things.

Gym subscription plans (feature flags enforced in the backend with `require_feature`, and returned from `/auth/me`):

Feature flag	Basic	Standard	Premium
members, plans, payments, checkin, staff, basic_reports	Yes	Yes	Yes
classes, personal_training, member_app, workouts, progress, churn_rules, crowd	No	Yes	Yes

Feature flag	Basic	Standard	Premium
ai_coach, ai_assistant, ask_gym, winback_agent, weekly_summary, multi_branch	No	No	Yes

Member limits: Basic 150, Standard 500, Premium unlimited (enforce on create). Locked features show a clean "Upgrade to unlock" card, never an error. A **dev-only** plan switcher is allowed only when `VITE_DEV_TOOLS=true`; it must not exist in a production build (prove it by searching `dist/`).

4. Data Model (Backend)

Create these tables (all with `id`, `gym_id`, `created_at`, `updated_at` unless noted):

- `gyms` (name, logo_url, phone, address, plan, plan_status, plan_renews_at, timezone, currency, settings JSON)
- `users` (name, email, phone, password_hash, role, pin_hash, active, language, linked `member_id` for member users)
- `members` (member_code unique per gym, name, phone, email, gender, dob, photo_url, emergency_contact, goal, joined_at, trainer_id, status_override, notes, leave_reason, deleted flag; never hard-delete)
- `health_forms` (member_id, par_q answers JSON, injuries text, conditions text, flagged bool, signed_name, signed_at)
- `membership_plans` (name, duration_days, price_cents, joining_fee_cents, max_freeze_days, active)
- `memberships` (member_id, plan_id, start_date, end_date, status stored or computed, frozen_days_used, auto_renew)
- `freezes` (membership_id, start, end, reason)
- `payments` (member_id, membership_id, amount_cents, method: cash/card/bank/online, reference, invoice_no sequential per gym, received_by, paid_at, note)
- `dues` (member_id, amount_cents, due_date, status open/paid, reason)
- `check_ins` (member_id, checked_in_at, checked_out_at, method qr/phone/manual, result ok/blocked, blocked_reason, source)
- `classes` (name, description, trainer_id, room, capacity, duration_min, active), `class_sessions` (class_id, starts_at, capacity, status), `bookings` (session_id, member_id, status booked/cancelled/attended/no_show/waitlist, created_at)

- `trainer_sessions` (trainer_id, member_id, starts_at, status), `session_packs` (member_id, total, used)
- `exercises` (name, muscle_group, equipment, level, how_to, media_url, contraindications tags)
- `workout_plans` (member_id, created_by, source manual/ai/template, status draft/pending_approval/approved/archived, title, days JSON, notes, approved_by, approved_at), `workout_logs` (member_id, plan_id, date, exercise_id, sets JSON)
- `meal_plans` (member_id, status same as workout plans, days JSON, notes)
- `measurements` (member_id, date, weight_kg, height_cm, waist_cm, chest_cm, hips_cm, notes), `goals` (member_id, type, target, due_date)
- `leads` (name, phone, source, status new/trial/joined/lost, next_follow_up, notes, score), `campaigns` (name, audience filter JSON, message, status)
- `messages` (member_or_lead ref, channel, body, status draft/approved/sent/failed, created_by, ai_generated bool, sent_at)
- `churn_scores` (member_id, score 0-100, level, reasons JSON, computed_at)
- `ai_actions` (user_id, feature, tool, input JSON, output JSON, status, cost_estimate, created_at)
- `settings` per gym (grace days, late-cancel hours, auto check-out hours, receipt header/footer)

Computed membership status (one function, fully tested): `active`, `expiring` (end within 7 days), `grace` (ended within grace days), `expired`, `frozen`. Renewal extends from the later of today and current end date. Freezing pushes the end date by the freeze days and respects `max_freeze_days`.

5. API (Backend, All Under `/api/v1`)

Standard error format `{ code, message, details }`. Pagination with `limit` and `cursor` or `page`. Auto docs at `/docs`. Export the OpenAPI file to `docs/openapi.json` with a script.

- **Auth:** `POST /auth/login` (email plus password), `POST /auth/pin-login` (gym code plus PIN, for front desk), `POST /auth/member-login` (phone plus code; in dev the code is always `123456`), `POST /auth/refresh`, `GET /auth/me` (user, role, gym, plan, feature flags, limits).

- **Members:** CRUD, search, `GET /members/{id}` (profile with status, visits, payments, plan, risk), `POST /members/{id}/freeze`, `POST /members/{id}/leave` (reason), health form endpoints, photo upload, Excel/CSV import (`POST /members/import` with a preview step) and export.
- **Plans and memberships:** `/membership-plans` CRUD, `POST /members/{id}/memberships` (new or renew), upgrade.
- **Payments:** `POST /payments`, `GET /payments`, `GET /dues`, `POST /dues/{id}/mark-paid`, `GET /payments/{id}/receipt`, daily cash summary.
- **Check-in:** `POST /checkin` (by member_code, phone or QR token; returns ok or blocked with a reason and member photo), `POST /checkin/{id}/checkout`, `GET /checkin/live` (people in gym now), `GET /me/qr-token` (a signed token that expires in 60 seconds; the member app refreshes it every 30 seconds so screenshots cannot be shared).
- **Classes:** classes, sessions, `POST /sessions/{id}/book`, cancel (late-cancel rule), waitlist promotion, mark attended.
- **Trainers:** my clients, sessions, packs.
- **Workouts:** exercises, templates, plans, approve or reject, logs. **Meal plans** similar. **Progress:** measurements, goals, streaks, personal records.
- **CRM:** leads, follow-ups, campaigns, messages (draft, approve, send, outbox).
- **Staff:** users CRUD, role, active, reset PIN.
- **Reports:** active members, new joins, renewals, expired, revenue by day and month, dues, peak hours, plan mix, trainer performance, class attendance. Each returns JSON, and `.csv` export.
- **Subscription:** `GET /subscription` (plan, renew date, invoices), a dev-only plan switch.
- **AI** (see Section 7).

6. Frontend Screens

Staff app (sidebar on tablet and desktop, bottom nav on phone; role-based menu; locked features show a lock and open an Upgrade card):

1. **Login** (email, and PIN pad for front desk) with language switch.
2. **Dashboard** (owner and manager): cards for active members, expiring this week, today's check-ins, today's revenue, dues total, people in gym now; revenue chart; **members at risk** card; an Ask Gym box.

3. **Check-in** (front desk home): one big search or scan box. Result fills the screen with **big green (OK) or red (blocked + reason)**, photo, name, plan, days left, and quick buttons: Renew, Collect Payment, Freeze. Live "in gym now" counter. Camera QR scan button. Must work in 2 taps.
4. **Members:** virtualized list with filters (status, plan, trainer, risk), add-member wizard (basic info, photo from webcam or upload, health form with on-screen signature, plan and first payment), member profile with tabs (Overview, Memberships, Payments, Visits, Workout, Progress, Messages, Health).
5. **Plans** (membership plan manager).
6. **Payments and Dues:** take payment dialog (big method buttons, quick amount chips), receipt print (80mm CSS and A4), WhatsApp bill link, dues list with Remind button (creates a message draft), daily cash close.
7. **Classes:** weekly timetable grid, session drawer (attendees, waitlist, mark attended), class manager.
8. **Trainers:** My Clients, approve AI plans, session calendar.
9. **Workouts:** exercise library, templates, plan builder.
10. **Leads (CRM):** simple board (New, Trial, Joined, Lost), follow-up reminders, convert to member.
11. **Messages:** drafts waiting for approval, outbox log, campaign builder.
12. **Reports:** report list with date range, table, small chart, CSV export, print.
13. **AI Center** (Premium): at-risk members, drafts, Ask Gym chat, weekly summary, AI Activity log.
14. **Staff** and **Settings** (gym profile, grace days, late-cancel hours, receipt text, language, theme, QR sheet for the gym door, sync status).
15. **My Plan (Subscription)** page with comparison cards.

Member app (`/m/*` , mobile-first PWA, bottom tabs: Home, Workout, Classes, Progress, Assistant):

- **Home:** big rotating QR, membership status and days left, streak, today's workout, "Best time to go" (crowd forecast), dues banner.
- **Workout:** today's exercises with how-to, log sets and reps with big steppers, rest timer. Shows only **approved** plans.
- **Classes:** timetable, book, cancel, waitlist.
- **Progress:** add weight and measurements, charts, goal progress, personal records.
- **Assistant:** chat with mic button (see AI 3).

- **Profile:** language, notifications, renew or pay (mock gateway button), install-app hint.

Offline check-in (front desk): cache active members (id, code, name, photo, status, end date) in IndexedDB. If the internet is down, check-in works from the cache and queues records. An `Online / Offline - N saved` badge shows. Sync on reconnect with duplicate protection (client-generated IDs).

7. AI Features (Build All Six; All Behind Feature Flags)

All AI endpoints live under `/api/v1/ai/*`, are logged in `ai_actions`, rate-limited, and have a fallback when the LLM is unavailable.

AI-1 Churn alert and win-back agent (flags: `churn_rules` for scores, `winback_agent` for drafts)

- A nightly job and an on-demand endpoint compute `churn_scores`. **Rule-based score 0-100**, easy to read: visits in last 14 days vs the member's own usual rate (biggest weight), days since last visit, membership ends within 14 days without renewal, open dues, missed booked classes, new member with fewer than 4 visits in first 21 days. Levels: High 60+, Medium 30-59, Low below 30. Each score stores **reasons** in plain words ("Visited 1 time in 14 days, usually 6").
- UI: at-risk list sorted by score, reasons shown, filter by trainer.
- `POST /ai/winback-draft {member_id}`: LLM writes a short, warm message in the member's preferred language, using the reasons and an optional offer chosen by staff. It is saved as a **draft** message. Staff edits and taps Send (outbox log plus `wa.me` link). **Never auto-send.**
- Leave a clean place in the code for a future ML model (`churn/model.py` interface) and record outcomes (did the member leave within 60 days) so it can be trained later. Show a small honest label: "Based on visit pattern".

AI-2 AI workout and meal coach (flag: `ai_coach`)

- Member app questionnaire: goal, level, days per week, session length, equipment, injuries or limits, food preference (veg or non-veg), budget level. Local food examples (rice, dhal, eggs, fish, chicken, vegetables, fruit, kottu portions).
- Backend builds a prompt with the **gym's exercise library (ids, equipment, level, contraindications)** and the member limits. LLM must return **JSON that matches a Pydantic schema**; exercise ids must exist in the library. If invalid: retry once, then use a deterministic template-based generator (so it always works).

- Safety: if the health form is flagged, or injuries are listed, the plan is built from safe exercises only and marked **"needs trainer review"** with a clear note. Every AI plan is saved as `pending_approval`. **The member sees nothing until a trainer approves** (trainer can edit exercises, sets, notes). Adaptive step: after 2 weeks of logs, `POST /ai/coach/adjust` suggests changes; trainer approves again.
- Show on plans: "General fitness guidance. Talk to a doctor if you have health problems." No medical advice, ever.

AI-3 Member assistant (chat and voice) (flag: `ai_assistant`)

- Chat UI in the member app (also a front-desk version later). Mic button uses the Web Speech API with a language selector; typed input always works.
- Tool-calling loop with these tools, all scoped to the **logged-in member only**: `get_my_membership`, `get_today_workout`, `list_classes`, `book_class`, `cancel_booking`, `get_my_dues`, `request_freeze`, `get_gym_hours`, `get_crowd_forecast`.
- Any action (book, cancel, freeze) shows a **confirm card** in the chat ("Book Zumba, Tue 6:00 PM?" with Confirm and Cancel). The action runs only after the member taps Confirm.
- Replies in the language the member used (EN, SI, TA). With `MockLLM`, support simple intent matching for demo (workout today, book class, dues, hours).

AI-4 Crowd and peak forecast (flag: `crowd`)

- Live count = check-ins without check-out in the last N hours (setting, default 3).
- Forecast = average visits by weekday and hour from history (use the seeded history), with a simple smoothing. Return a 0-100 "busy level" and a label (Quiet, Normal, Busy).
- UI: member Home shows "Best time to go: today 2-4 PM" and a small hour bar chart; staff dashboard shows a busy-by-hour chart plus a plain suggestion ("Add a trainer 6-8 PM").

AI-5 Ask Gym (owner assistant) (flag: `ask_gym`)

- Chat for owner and manager. **No raw SQL from the LLM.** Use fixed tools that run safe queries: `revenue_summary(range)`, `compare_revenue(range_a, range_b)`, `active_members()`, `new_vs_lost(range)`, `churn_risk_summary()`, `dues_summary()`, `top_classes(range)`, `peak_hours()`, `plan_mix()`, `trainer_performance(range)`.
- Answer in simple words plus an optional small chart (return chart data with the text). Examples to support: "How is revenue vs last month?", "How many members are at risk?", "Which class is most popular?", "Who owes the most?". Voice input like AI-3.

AI-6 Weekly summary (flag: `weekly_summary`)

- `GET /ai/weekly-summary`: this week vs last week (revenue, new joins, losses, check-ins), top and weakest class, members at risk, dues, and 2-3 plain-words tips. Store it. Button "Send to owner" (outbox, `wa.me` link). A scheduled job generates it every Sunday evening.

AI Activity log page lists `ai_actions` (what, who, when, result) and shows approve or undo where possible.

8. Seed Data (Make the Demo Feel Real)

Create `python -m app.seed` that fills a demo gym **"Iron Peak Fitness"** (Premium plan):

- 5 staff logins: owner, manager, front desk (PIN 1234), 2 trainers. Dev password `Pass1234!`, shown only in README and a dev-only login helper.
- **120 members** with realistic Sri Lankan and mixed names, photos as initials avatars, joined across the last 18 months, with a realistic mix: about 60% active, 10% expiring, 8% grace, 15% expired, 4% frozen, 3% stopped.
- **90 days of check-in history** with realistic patterns (peaks 6-8 AM and 5-8 PM, quieter weekends), regulars vs occasional vs fading members (about 12 members clearly fading, so the churn list is interesting).
- Plans: Day Pass Rs 800, Monthly Rs 5,000, Student Rs 4,000, Quarterly Rs 13,500, Yearly Rs 48,000, Couple Monthly Rs 8,500.
- Payment history, about 8 open dues, 8 classes with a weekly timetable and some bookings, 40 exercises (with equipment, level, muscle group, contraindication tags), 4 workout templates, 30 leads, 3 member users who can log in with phone and code `123456`, and one approved plan plus one pending AI plan.
- A second tiny demo gym with 5 members, used only by the tenant-isolation tests.

9. Stages (Work in This Order)

Day 1

- **Stage 0 (setup):** repo, folders, `docker-compose.yml` (PostgreSQL optional), `.env.example`, linting, formatting, CI file, README with run steps, `docs/DECISIONS.md`. Backend and frontend both start and show a hello page.
- **Stage 1 (backend core):** database models, auth (all three logins plus refresh), roles, tenant isolation, plan gating, members, membership plans and memberships with the status function, payments, dues, receipts, check-in with the QR token, staff, reports,

seed script. Tests: status function, renewal and freeze, payments, QR token expiry, tenant isolation, role permissions, plan gating, member limit. Export `docs/openapi.json`.

- **Stage 2 (staff app core):** design tokens, base components (`BigButton`, `StatusPill`, `MoneyText`, `Skeleton`, `EmptyState`, `ConfirmDialog`, `PageHeader`, `UpgradeCard`), app shell with page transitions, i18n setup (EN, SI, TA), generated API types, login, dashboard, **check-in screen**, members (list, add wizard, profile), plans, payments and dues, receipts, staff, basic reports, settings. All against the real backend.
- **Stage 3 (classes, trainers, workouts, CRM, member app):** backend and frontend for classes and bookings, trainer sessions, exercises, workout plans and logs, progress, leads, messages and outbox. Member app screens (Home with rotating QR, Workout, Classes, Progress, Profile) as a PWA.

Day 2

- **Stage 4 (AI heroes):** AI-1 to AI-6 exactly as in Section 7, plus the AI Center page, the member Assistant tab, the crowd forecast on both apps, and the AI Activity log. Build `LLMClient` with `MockLLM` first so everything works without a key, then `GeminiClient`. Tests: churn score cases, JSON validation and fallback for the coach, tool permission scoping (a member cannot read another member's data), confirm-before-action flow, crowd forecast shape.
- **Stage 5 (SaaS, offline, polish):** Subscription page and plan states (grace period amber banner, read-only red banner with one shared `useReadOnly` hook), dev-only plan switcher (stripped from production), offline check-in cache and sync, dark mode toggle (Light, Dark, Auto), complete i18n for navigation, login, check-in, members, payments, member app and AI chat (report the true percent translated for SI and TA; add Noto Sans Sinhala and Tamil fonts), print CSS for receipts, responsive check at 1280, 1024, 768 and 390 px, accessibility check (labels, focus ring, contrast), speed work (route lazy loading, `manualChunks`, named icon imports so the main bundle stays under 300 kB or 100 kB gzip).
- **Stage 6 (tests, docs, demo):** Backend tests (at least 40 in total), frontend unit tests (at least 15), and 3 Playwright flows if time allows: (a) front desk checks in an active member and blocks an expired one, (b) renew membership and take payment, (c) member books a class from the assistant with confirm. Write `docs/DEMO.md` (the 3-minute demo script below), `docs/FRONTEND_STATUS.md`, `docs/AI_FEATURES.md` (how each AI feature works, limits, fallbacks), `docs/SECURITY_TODO.md` (honest list of what is still needed before production), and update README.

10. The 3-Minute Demo (Make It Work with Seed Data)

1. Front desk scans or types a member: **green check in 1 second**, live counter goes up. An expired member shows **red with the reason** and a Renew button.
2. Owner dashboard: **"12 members at risk"**. Open one: reasons shown. Tap "Draft win-back": AI writes a message. Tap Send (outbox plus WhatsApp link).
3. Member app: a new member answers 6 questions, AI creates a workout plan, trainer approves in one tap, member sees it.
4. Member says or types "Book me Zumba tomorrow 6pm": confirm card, tap Confirm, booking appears on the staff timetable.
5. Owner asks "How is revenue vs last month?": answer plus chart. Weekly summary is ready to send.

11. Definition of Done

- Backend and frontend run with one command each (documented), seed works, `/docs` works.
- All tests, lint, type-check and build pass (show the outputs).
- The demo above works end to end with `MockLLM` (no API key) and with `GeminiClient` when a key is set.
- Tenant isolation, role checks, plan gating and member-only AI tools are all covered by tests.
- A production build contains no dev plan switcher, no demo logins, and no mock-only code paths exposed.
- Every screen has loading, empty and error states, works on a 390 px phone, and uses i18n.
- Docs are written.

12. Final Report (Send This When Finished)

For each Stage: what you built, what you tested (with counts), what is mocked, what is real, known bugs, shortcuts you took, and the real percent translated for Sinhala and Tamil. Then give the exact run commands, all test logins, and a 10-step click path I can follow to check everything. Be honest about anything not finished.

END OF PROMPT

Short "Continue" Prompts

Use these if the agent stops or loses context.

Continue from a Stage:

```
Continue the Ralypto Gym build. First read docs/DECISIONS.md, README.md and
git log to see what is done. Do not redo finished work. Continue with Stage
<N> exactly as in the master prompt, then run lint, type-check, tests and
build, and commit. Report honestly what works.
```

Check the work:

```
Give me an honest status report of Ralypto Gym. Run backend tests, frontend
tests, lint, type-check and build and show the output. List every screen and
API with status (Done / Partly / Placeholder / Not started), what is mocked,
what is real, bugs, and the next 10 tasks. Do not say anything is done unless
you ran it.
```

What To Check Yourself (10 Minutes Each Day)

Day 1: log in as front desk, check in an active and an expired member, renew one, take a payment, print a receipt, try a Basic-plan gym and see the locks.

Day 2: open the at-risk list, draft a win-back message, create an AI workout plan and approve it as a trainer, book a class with the assistant, switch the phone width to 390 px, turn off the internet and check in a member.

After the 2 Days

1. Show it to 2-3 gym owners and one trainer. Ask what is missing or confusing.
2. Pick one pilot gym and import their real members from Excel.
3. Add a real payment gateway, a real SMS or WhatsApp provider, hosting, daily backups, and a security review (see `docs/SECURITY_TODO.md`).
4. Check the local data-protection law for health data and member photos.